Operating Systems Lab

C Program Listings

Contents

2(a) fork() - Parent and Child Process

2(b) exec - Display Current Date and Time

2(c) stat() - File Information

2(d) fork(), wait() - Process Creation

2(e) File System Calls - creat, open, write, read, close

3(a) First Come First Serve (FCFS) Scheduling

3(b) Shortest Job First (SJF) Scheduling

3(c) Round Robin Scheduling

3(d) Priority Scheduling

4(a) Shared Memory - Writer Process

4(b) Shared Memory - Reader Process

5(a), 5(b) Banker's Algorithm - Deadlock Avoidance

6(a) Page Replacement - LRU

6(b) Page Replacement - Optimal

6(c) Page Replacement - FIFO

7(a) Memory Management - Best Fit

7(b) Memory Management - First Fit

7(c) Memory Management - Worst Fit

8(a) Disk Scheduling - FCFS

8(b) Disk Scheduling - SSTF

8(c) Disk Scheduling - SCAN

9 Access Control - DAC, RBAC, MAC (Code)

10 File I/O - Text and Binary Files

2(a) fork() - Parent and Child Process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process");
        printf("\nChild PID = %d", getpid());
        printf("\nParent PID = %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("\nParent Process");
        printf("\nParent PID = %d", getpid());
        printf("\nChild PID = %d\n", pid);
    }

    return 0;
}
```

2(b) exec - Display Current Date and Time

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Displaying Current Date and Time\n");

    execlp("date", "date", NULL);

    printf("This line will not execute.\n");

    return 0;
}
```

2(c) stat() - File Information

```
#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>
#include <time.h>

int main()
{
    struct stat s;
    char filename[100];

    printf("Enter the file name: ");
    scanf("%s", filename);

    if (stat(filename, &s) == -1)
    {
        perror("stat");
        return 1;
    }

    printf("\nFile Information\n");
    printf("----------------------------\n");
    printf("File Size          : %ld bytes\n", s.st_size);
    printf("Inode Number       : %ld\n", s.st_ino);
    printf("File Mode          : %o\n", s.st_mode);
    printf("Number of Links    : %ld\n", s.st_nlink);
    printf("User ID            : %d\n", s.st_uid);
    printf("Group ID           : %d\n", s.st_gid);
    printf("Last Access Time   : %s", ctime(&s.st_atime));
    printf("Last Modified Time : %s", ctime(&s.st_mtime));

    return 0;
}
```

2(d) fork(), wait() - Process Creation

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before Fork\n");

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process Executing...\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);

        printf("\nParent Process Resumed...\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }

    return 0;
}
```

2(e) File System Calls - creat, open, write, read, close

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main()
{
    int fd;
    char write_buf[] = "Operating Systems Lab";
    char read_buf[50];

    /* Create and open file */
    fd = creat("sample.txt", 0666);

    if (fd < 0)
    {
        printf("File creation failed\n");
        return 1;
    }

    close(fd);

    /* Open file in write mode */
    fd = open("sample.txt", O_WRONLY);

    write(fd, write_buf, strlen(write_buf));

    close(fd);

    /* Open file in read mode */
    fd = open("sample.txt", O_RDONLY);

    read(fd, read_buf, sizeof(read_buf));

    printf("Contents of the file: %s\n", read_buf);

    close(fd);

    return 0;
}
```

3(a) First Come First Serve (FCFS) Scheduling

```
#include <stdio.h>

int main()
{
    int n, i;
    int pid[20], st[20];
    int wt[20], tat[20];
    int total_wt = 0, total_tat = 0;
    float avg_wt, avg_tat;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("\nEnter Process ID and Service Time:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d ID: ", i + 1);
        scanf("%d", &pid[i]);

        printf("Service Time: ");
        scanf("%d", &st[i]);
    }

    wt[0] = 0;
    tat[0] = st[0];

    total_wt += wt[0];
    total_tat += tat[0];

    for(i = 1; i < n; i++)
    {
        wt[i] = tat[i - 1];
        tat[i] = wt[i] + st[i];

        total_wt += wt[i];
        total_tat += tat[i];
    }

    avg_wt = (float)total_wt / n;
    avg_tat = (float)total_tat / n;

    printf("\n-----------------------------------------------------------------\n");
    printf("PID\tService Time\tWaiting Time\tTurnaround Time\n");
    printf("-----------------------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t\t%d\t\t%d\n", pid[i], st[i], wt[i], tat[i]);
    }

    printf("-----------------------------------------------------------------\n");
    printf("Average Waiting Time    = %.2f\n", avg_wt);
    printf("Average Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

3(b) Shortest Job First (SJF) Scheduling

```c
#include <stdio.h>

int main()
{
    int n, i, j;
    int pid[20], st[20];
    int wt[20], tat[20];
    int temp;
    int total_wt = 0, total_tat = 0;
    float avg_wt, avg_tat;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("\nEnter Process ID and Service Time:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d ID: ", i + 1);
        scanf("%d", &pid[i]);

        printf("Service Time: ");
        scanf("%d", &st[i]);
    }

    /* Sort processes according to Service Time */
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(st[i] > st[j])
            {
                temp = st[i];
                st[i] = st[j];
                st[j] = temp;

                temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }

    wt[0] = 0;
    tat[0] = st[0];

    total_wt = wt[0];
    total_tat = tat[0];

    for(i = 1; i < n; i++)
    {
        wt[i] = tat[i - 1];
        tat[i] = wt[i] + st[i];

        total_wt += wt[i];
        total_tat += tat[i];
    }

    avg_wt = (float)total_wt / n;
    avg_tat = (float)total_tat / n;

    printf("\n----------------------------------------------------------------\n");
    printf("PID\tService Time\tWaiting Time\tTurnaround Time\n");
    printf("----------------------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t\t%d\t\t%d\n",
               pid[i], st[i], wt[i], tat[i]);
    }

    printf("----------------------------------------------------------------\n");
    printf("Average Waiting Time    = %.2f\n", avg_wt);
    printf("Average Turnaround Time = %.2f\n", avg_tat);
```

```
    return 0;
}
```

3(c) Round Robin Scheduling

```
#include <stdio.h>

int main()
{
    int i, n, tq;
    int pid[20], bt[20], rt[20];
    int wt[20], tat[20];
    int time = 0, remain, flag = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    remain = n;

    printf("\nEnter Process ID and Burst Time:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d ID: ", i + 1);
        scanf("%d", &pid[i]);

        printf("Burst Time: ");
        scanf("%d", &bt[i]);

        rt[i] = bt[i];
    }

    printf("\nEnter Time Quantum: ");
    scanf("%d", &tq);

    printf("\n----------------------------------------------------------------\n");
    printf("PID\tBurst Time\tWaiting Time\tTurnaround Time\n");
    printf("----------------------------------------------------------------\n");

    for(i = 0; remain != 0; )
    {
        if(rt[i] <= tq && rt[i] > 0)
        {
            time += rt[i];
            rt[i] = 0;
            flag = 1;
        }
        else if(rt[i] > 0)
        {
            rt[i] -= tq;
            time += tq;
        }

        if(flag == 1 && rt[i] == 0)
        {
            remain--;

            tat[i] = time;
            wt[i] = tat[i] - bt[i];

            avg_wt += wt[i];
            avg_tat += tat[i];

            printf("%d\t%d\t\t%d\t\t%d\n",
                   pid[i], bt[i], wt[i], tat[i]);

            flag = 0;
        }

        if(i == n - 1)
            i = 0;
        else
            i++;
    }

    printf("----------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avg_wt / n);
    printf("Average Turnaround Time = %.2f\n", avg_tat / n);
```

```
    return 0;
}
```

3(d) Priority Scheduling

```
#include <stdio.h>

struct process
{
    int pid;
    int bt;
    int pri;
    int wt;
    int tat;
};

int main()
{
    struct process p[20], temp;
    int n, i, j;
    int total_wt = 0, total_tat = 0;
    float avg_wt, avg_tat;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("\nEnter Process ID, Burst Time and Priority:\n");

    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);

        printf("Process ID : ");
        scanf("%d", &p[i].pid);

        printf("Burst Time : ");
        scanf("%d", &p[i].bt);

        printf("Priority (Lower number = Higher Priority): ");
        scanf("%d", &p[i].pri);
    }

    /* Sort according to Priority */
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(p[i].pri > p[j].pri)
            {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    p[0].wt = 0;
    p[0].tat = p[0].bt;

    total_wt = p[0].wt;
    total_tat = p[0].tat;

    for(i = 1; i < n; i++)
    {
        p[i].wt = p[i-1].tat;
        p[i].tat = p[i].wt + p[i].bt;

        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }

    avg_wt = (float)total_wt / n;
    avg_tat = (float)total_tat / n;

    printf("\n-----------------------------------------------------------------\n");
    printf("PID\tBT\tPriority\tWT\tTAT\n");
    printf("-----------------------------------------------------------------\n");
```

```
    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t\t%d\t%d\n",
               p[i].pid,
               p[i].bt,
               p[i].pri,
               p[i].wt,
               p[i].tat);
    }

    printf("-------------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avg_wt);
    printf("Average Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

4(a) Shared Memory - Writer Process

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main()
{
    // ftok to generate unique key
    key_t key = ftok("shmfile",65);

    // shmget returns an identifier in shmid
    int shmid = shmget(key,1024,0666|IPC_CREAT);

    // shmat to attach to shared memory
    char *str = (char*) shmat(shmid,(void*)0,0);

    printf("Write Data : ");
    gets(str);

    printf("Data written in memory: %s\n",str);

    //detach from shared memory
    shmdt(str);

    return 0;
}
```

4(b) Shared Memory - Reader Process

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main()
{
    // ftok to generate unique key
    key_t key = ftok("shmfile",65);

    // shmget returns an identifier in shmid
    int shmid = shmget(key,1024,0666|IPC_CREAT);

    // shmat to attach to shared memory
    char *str = (char*) shmat(shmid,(void*)0,0);

    printf("Data read from memory: %s\n",str);

    //detach from shared memory
    shmdt(str);

    // destroy the shared memory
    shmctl(shmid,IPC_RMID,NULL);

    return 0;
}
```

5(a), 5(b) Banker's Algorithm - Deadlock Avoidance

```
#include <stdio.h>

int main()
{
    // P0, P1, P2, P3, P4 are the Process names here
    int n, m, i, j, k;
    n = 5; // Number of processes
    m = 3; // Number of resources

    int alloc[5][3] = { { 0, 1, 0 }, // P0 // Allocation Matrix
                        { 2, 0, 0 }, // P1
                        { 3, 0, 2 }, // P2
                        { 2, 1, 1 }, // P3
                        { 0, 0, 2 } }; // P4

    int max[5][3] = { { 7, 5, 3 }, // P0 // MAX Matrix
                      { 3, 2, 2 }, // P1
                      { 9, 0, 2 }, // P2
                      { 2, 2, 2 }, // P3
                      { 4, 3, 3 } }; // P4

    int avail[3] = { 3, 3, 2 }; // Available Resources

    int f[n], ans[n], ind = 0;
    for (k = 0; k < n; k++) {
        f[k] = 0;
    }

    int need[n][m];
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }

    int y = 0;
    for (k = 0; k < 5; k++) {
        for (i = 0; i < n; i++) {
            if (f[i] == 0) {
                int flag = 0;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]){
                        flag = 1;
                        break;
                    }
                }

                if (flag == 0) {
                    ans[ind++] = i;
                    for (y = 0; y < m; y++)
                        avail[y] += alloc[i][y];
                    f[i] = 1;
                }
            }
        }
    }

    printf("Following is the SAFE Sequence\n");
    for (i = 0; i < n - 1; i++)
        printf(" P%d ->", ans[i]);
    printf(" P%d", ans[n - 1]);

    return (0);
}
```

6(a) Page Replacement - LRU

```
#include <stdio.h>

int main()
{
    int frames[10], pages[50], counter[10];
    int n, f, i, j, k;
    int page_faults = 0, page_hits = 0;
    int time = 0, found, pos;

    printf("Enter the number of frames: ");
    scanf("%d", &f);

    printf("Enter the number of pages: ");
    scanf("%d", &n);

    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    /* Initialize frames */
    for (i = 0; i < f; i++)
    {
        frames[i] = -1;
        counter[i] = 0;
    }

    printf("\nPage\tFrames\t\tStatus\n");

    for (i = 0; i < n; i++)
    {
        found = 0;

        /* Check whether page is already present */
        for (j = 0; j < f; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                time++;
                counter[j] = time;
                page_hits++;
                break;
            }
        }

        /* Page fault */
        if (found == 0)
        {
            page_faults++;
            time++;

            /* Check for empty frame */
            pos = -1;
            for (j = 0; j < f; j++)
            {
                if (frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }

            /* If no empty frame, find LRU page */
            if (pos == -1)
            {
                pos = 0;

                for (j = 1; j < f; j++)
                {
                    if (counter[j] < counter[pos])
                        pos = j;
                }
            }
```

```
            frames[pos] = pages[i];
            counter[pos] = time;
        }

        /* Display frames */
        printf("%d\t", pages[i]);

        for (k = 0; k < f; k++)
        {
            if (frames[k] == -1)
                printf("- ");
            else
                printf("%d ", frames[k]);
        }

        if (found)
            printf("\tHit");
        else
            printf("\tPage Fault");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d", page_faults);
    printf("\nTotal Page Hits   = %d\n", page_hits);

    return 0;
}
```

6(b) Page Replacement - Optimal

```
#include <stdio.h>

int isPresent(int cache[], int cacheSize, int page)
{
    int i;

    for (i = 0; i < cacheSize; i++)
    {
        if (cache[i] == page)
            return 1;
    }

    return 0;
}

/* Find how far a page is used in the future */
int findLength(int ref[], int n, int current, int page)
{
    int i;

    for (i = current + 1; i < n; i++)
    {
        if (ref[i] == page)
            return i - current;
    }

    /* Page is not used again */
    return 9999;
}

/* Find the page with maximum future distance */
int findMax(int length[], int cacheSize)
{
    int i, max = 0;

    for (i = 1; i < cacheSize; i++)
    {
        if (length[i] > length[max])
            max = i;
    }

    return max;
}

int main()
{
    int ref[50], cache[10], length[10];
    int n, cacheSize;
    int i, j, index;
    int pageFaults = 0;

    printf("Enter the number of pages: ");
    scanf("%d", &n);

    printf("Enter the reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    printf("Enter the cache size: ");
    scanf("%d", &cacheSize);

    /* Initialize cache */
    for (i = 0; i < cacheSize; i++)
        cache[i] = -1;

    printf("\nPage\tCache\t\tStatus\n");

    for (i = 0; i < n; i++)
    {
        /* Page is already present */
        if (isPresent(cache, cacheSize, ref[i]))
        {
            printf("%d\t", ref[i]);
```

```
            for (j = 0; j < cacheSize; j++)
                printf("%d ", cache[j]);

            printf("\tHit\n");
        }
        else
        {
            /* Fill empty cache first */
            index = -1;

            for (j = 0; j < cacheSize; j++)
            {
                if (cache[j] == -1)
                {
                    index = j;
                    break;
                }
            }

            /* If cache is full, find optimal replacement */
            if (index == -1)
            {
                for (j = 0; j < cacheSize; j++)
                {
                    length[j] = findLength(ref, n, i, cache[j]);
                }

                index = findMax(length, cacheSize);
            }

            cache[index] = ref[i];
            pageFaults++;

            printf("%d\t", ref[i]);

            for (j = 0; j < cacheSize; j++)
                printf("%d ", cache[j]);

            printf("\tPage Fault\n");
        }
    }

    printf("\nTotal Page Faults = %d\n", pageFaults);

    return 0;
}
```

6(c) Page Replacement - FIFO

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f;
    int i, j;
    int pointer = 0;
    int pageFaults = 0, pageHits = 0;
    int found;

    printf("Enter the number of pages: ");
    scanf("%d", &n);

    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter the number of frames: ");
    scanf("%d", &f);

    /* Initialize frames */
    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");

    for (i = 0; i < n; i++)
    {
        found = 0;

        /* Check whether page is already in memory */
        for (j = 0; j < f; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                pageHits++;
                break;
            }
        }

        /* Page fault */
        if (found == 0)
        {
            frames[pointer] = pages[i];

            /* Move pointer in circular manner */
            pointer = (pointer + 1) % f;

            pageFaults++;
        }

        /* Display current page and frames */
        printf("%d\t", pages[i]);

        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        if (found)
            printf("\tHit");
        else
            printf("\tPage Fault");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d", pageFaults);
```

```
    printf("\nTotal Page Hits   = %d\n", pageHits);

    return 0;
}
```

7(a) Memory Management - Best Fit

```
/*
 * Memory Management Scheme using Best Fit
 * ---------------------------------------
 * Allocates each process to the smallest block that is large enough
 * to hold it (i.e., the block that leaves the least fragmentation).
 */

#include <stdio.h>

int main() {
    int nBlocks, nProcesses;
    int blockSize[50], processSize[50];
    int allocation[50];      // allocation[i] = block index allocated to process i (-1 if none)
    int blockFree[50];       // 1 = block still free, 0 = block used
    int i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nBlocks);

    printf("Enter sizes of the %d blocks:\n", nBlocks);
    for (i = 0; i < nBlocks; i++) {
        printf("Block %d size: ", i + 1);
        scanf("%d", &blockSize[i]);
        blockFree[i] = 1;
    }

    printf("Enter number of processes: ");
    scanf("%d", &nProcesses);

    printf("Enter sizes of the %d processes:\n", nProcesses);
    for (i = 0; i < nProcesses; i++) {
        printf("Process %d size: ", i + 1);
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }

    /* Best Fit allocation logic */
    for (i = 0; i < nProcesses; i++) {
        int bestIdx = -1;

        for (j = 0; j < nBlocks; j++) {
            if (blockFree[j] && blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockFree[bestIdx] = 0;   // mark block as used
        }
    }

    /* Display results */
    printf("\nProcess No.\tProcess Size\tBlock No.\tFragmentation\n");
    for (i = 0; i < nProcesses; i++) {
        printf("P%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1) {
            printf("B%d\t\t%d\n", allocation[i] + 1,
                   blockSize[allocation[i]] - processSize[i]);
        } else {
            printf("Not Allocated\t-\n");
        }
    }

    return 0;
}
```

7(b) Memory Management - First Fit

```
/*
 * Memory Management Scheme using First Fit
 * ----------------------------------------
 * Allocates each process to the first free block that is large enough
 * to hold it, scanning blocks in order.
 */

#include <stdio.h>

int main() {
    int nBlocks, nProcesses;
    int blockSize[50], processSize[50];
    int allocation[50];      // allocation[i] = block index allocated to process i (-1 if none)
    int blockFree[50];       // 1 = block still free, 0 = block used
    int i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nBlocks);

    printf("Enter sizes of the %d blocks:\n", nBlocks);
    for (i = 0; i < nBlocks; i++) {
        printf("Block %d size: ", i + 1);
        scanf("%d", &blockSize[i]);
        blockFree[i] = 1;
    }

    printf("Enter number of processes: ");
    scanf("%d", &nProcesses);

    printf("Enter sizes of the %d processes:\n", nProcesses);
    for (i = 0; i < nProcesses; i++) {
        printf("Process %d size: ", i + 1);
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }

    /* First Fit allocation logic */
    for (i = 0; i < nProcesses; i++) {
        for (j = 0; j < nBlocks; j++) {
            if (blockFree[j] && blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockFree[j] = 0;   // mark block as used
                break;              // stop at the FIRST block that fits
            }
        }
    }

    /* Display results */
    printf("\nProcess No.\tProcess Size\tBlock No.\tFragmentation\n");
    for (i = 0; i < nProcesses; i++) {
        printf("P%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1) {
            printf("B%d\t\t%d\n", allocation[i] + 1,
                   blockSize[allocation[i]] - processSize[i]);
        } else {
            printf("Not Allocated\t-\n");
        }
    }

    return 0;
}
```

7(c) Memory Management - Worst Fit

```
/*
 * Memory Management Scheme using Worst Fit
 * ----------------------------------------
 * Allocates each process to the LARGEST free block that is big enough
 * to hold it, leaving the biggest possible leftover space.
 */

#include <stdio.h>

int main() {
    int nBlocks, nProcesses;
    int blockSize[50], processSize[50];
    int allocation[50];      // allocation[i] = block index allocated to process i (-1 if none)
    int blockFree[50];       // 1 = block still free, 0 = block used
    int i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nBlocks);

    printf("Enter sizes of the %d blocks:\n", nBlocks);
    for (i = 0; i < nBlocks; i++) {
        printf("Block %d size: ", i + 1);
        scanf("%d", &blockSize[i]);
        blockFree[i] = 1;
    }

    printf("Enter number of processes: ");
    scanf("%d", &nProcesses);

    printf("Enter sizes of the %d processes:\n", nProcesses);
    for (i = 0; i < nProcesses; i++) {
        printf("Process %d size: ", i + 1);
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }

    /* Worst Fit allocation logic */
    for (i = 0; i < nProcesses; i++) {
        int worstIdx = -1;

        for (j = 0; j < nBlocks; j++) {
            if (blockFree[j] && blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockFree[worstIdx] = 0;   // mark block as used
        }
    }

    /* Display results */
    printf("\nProcess No.\tProcess Size\tBlock No.\tFragmentation\n");
    for (i = 0; i < nProcesses; i++) {
        printf("P%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1) {
            printf("B%d\t\t%d\n", allocation[i] + 1,
                   blockSize[allocation[i]] - processSize[i]);
        } else {
            printf("Not Allocated\t-\n");
        }
    }

    return 0;
}
```

8(a) Disk Scheduling - FCFS

```
#include <stdio.h>
#include <stdlib.h>

void fcfs(int requests[], int n, int head) {
    int total = 0, current = head;

    printf("\n--- FCFS Disk Scheduling ---\n");
    printf("Seek Sequence: %d", head);

    for (int i = 0; i < n; i++) {
        total += abs(requests[i] - current);
        current = requests[i];
        printf(" -> %d", current);
    }

    printf("\nTotal Seek Time (FCFS): %d cylinders\n", total);
}

int main() {
    int n, head;
    int requests[100];

    printf("Enter the number of requests: ");
    scanf("%d", &n);

    printf("Enter the request queue: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter the initial head position: ");
    scanf("%d", &head);

    fcfs(requests, n, head);

    return 0;
}
```

8(b) Disk Scheduling - SSTF

```
#include <stdio.h>
#include <stdlib.h>

void sstf(int requests[], int n, int head) {
    int visited[100] = {0};
    int total = 0, current = head;

    printf("\n--- SSTF Disk Scheduling ---\n");
    printf("Seek Sequence: %d", head);

    for (int count = 0; count < n; count++) {
        int minDist = 100000, index = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i] && abs(requests[i] - current) < minDist) {
                minDist = abs(requests[i] - current);
                index = i;
            }
        }

        visited[index] = 1;
        total += minDist;
        current = requests[index];
        printf(" -> %d", current);
    }

    printf("\nTotal Seek Time (SSTF): %d cylinders\n", total);
}

int main() {
    int n, head;
    int requests[100];

    printf("Enter the number of requests: ");
    scanf("%d", &n);

    printf("Enter the request queue: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter the initial head position: ");
    scanf("%d", &head);

    sstf(requests, n, head);

    return 0;
}
```

8(c) Disk Scheduling - SCAN

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

void scan(int requests[], int n, int head, int disk_size, int direction) {
    int total = 0, current = head;
    int sorted[100];

    for (int i = 0; i < n; i++)
        sorted[i] = requests[i];
    sorted[n] = head;
    qsort(sorted, n + 1, sizeof(int), compare);

    int pos = 0;
    for (int i = 0; i <= n; i++)
        if (sorted[i] == head) { pos = i; break; }

    printf("\n--- SCAN Disk Scheduling ---\n");
    printf("Direction: %s\n", direction == 1 ? "Towards higher cylinders" : "Towards lower cylinders");
    printf("Seek Sequence: %d", head);

    if (direction == 1) {
        /* Move right to the end of disk, then reverse */
        for (int i = pos + 1; i <= n; i++) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
        if (current != disk_size - 1) {
            total += abs((disk_size - 1) - current);
            current = disk_size - 1;
            printf(" -> %d", current);
        }
        for (int i = pos - 1; i >= 0; i--) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
    } else {
        /* Move left to the start of disk, then reverse */
        for (int i = pos - 1; i >= 0; i--) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
        if (current != 0) {
            total += abs(current - 0);
            current = 0;
            printf(" -> %d", current);
        }
        for (int i = pos + 1; i <= n; i++) {
            total += abs(sorted[i] - current);
            current = sorted[i];
            printf(" -> %d", current);
        }
    }

    printf("\nTotal Seek Time (SCAN): %d cylinders\n", total);
}

int main() {
    int n, head, disk_size, direction;
    int requests[100];

    printf("Enter the maximum number of cylinders: ");
    scanf("%d", &disk_size);

    printf("Enter the number of requests: ");
    scanf("%d", &n);
```

```
    printf("Enter the request queue: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter the initial head position: ");
    scanf("%d", &head);

    printf("Enter direction (1 = towards higher, 0 = towards lower): ");
    scanf("%d", &direction);

    scan(requests, n, head, disk_size, direction);

    return 0;
}
```

9 Access Control - DAC, RBAC, MAC (Code)

```
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>

/* ---------- DAC: check file permissions ---------- */
void check_dac(const char *filename) {
    struct stat st;
    if (stat(filename, &st) != 0) {
        printf("File not found: %s\n", filename);
        return;
    }
    printf("\n[DAC] File: %s\n", filename);
    printf("Owner : %c%c%c\n",
           (st.st_mode & S_IRUSR) ? 'r' : '-',
           (st.st_mode & S_IWUSR) ? 'w' : '-',
           (st.st_mode & S_IXUSR) ? 'x' : '-');
    printf("Group : %c%c%c\n",
           (st.st_mode & S_IRGRP) ? 'r' : '-',
           (st.st_mode & S_IWGRP) ? 'w' : '-',
           (st.st_mode & S_IXGRP) ? 'x' : '-');
    printf("Others: %c%c%c\n",
           (st.st_mode & S_IROTH) ? 'r' : '-',
           (st.st_mode & S_IWOTH) ? 'w' : '-',
           (st.st_mode & S_IXOTH) ? 'x' : '-');
}

/* ---------- RBAC: role-based permission check ---------- */
struct Role {
    char name[10];
    int can_read;
    int can_write;
    int can_execute;
};

void check_rbac(const char *role_name, const char *operation) {
    struct Role roles[] = {
        {"admin", 1, 1, 1},
        {"staff", 1, 1, 0},
        {"guest", 1, 0, 0}
    };
    int n = sizeof(roles) / sizeof(roles[0]);

    printf("\n[RBAC] Role: %s, Operation: %s\n", role_name, operation);
    for (int i = 0; i < n; i++) {
        if (strcmp(roles[i].name, role_name) == 0) {
            int allowed = 0;
            if (strcmp(operation, "read") == 0)    allowed = roles[i].can_read;
            if (strcmp(operation, "write") == 0)   allowed = roles[i].can_write;
            if (strcmp(operation, "execute") == 0) allowed = roles[i].can_execute;
            printf("Access %s\n", allowed ? "GRANTED" : "DENIED");
            return;
        }
    }
    printf("Unknown role\n");
}

/* ---------- MAC: label-based check (Bell-LaPadula style) ---------- */
/* Levels: 1 = public, 2 = confidential, 3 = secret */
void check_mac(int user_level, int file_level, const char *operation) {
    int allowed = 0;
    printf("\n[MAC] User level: %d, File level: %d, Operation: %s\n",
           user_level, file_level, operation);
    if (strcmp(operation, "read") == 0)
        allowed = (user_level >= file_level);   /* no read up */
    else if (strcmp(operation, "write") == 0)
        allowed = (user_level <= file_level);   /* no write down */
    printf("Access %s\n", allowed ? "GRANTED" : "DENIED");
}

int main() {
    /* DAC: create a test file and set its permissions first */
    FILE *fp = fopen("testfile.txt", "w");
    if (fp) { fprintf(fp, "sample data\n"); fclose(fp); }
```

```
    system("chmod 754 testfile.txt");   /* owner rwx, group r-x, others r-- */
    check_dac("testfile.txt");

    /* RBAC */
    check_rbac("admin", "write");
    check_rbac("guest", "write");
    check_rbac("staff", "execute");

    /* MAC */
    check_mac(3, 2, "read");    /* secret user reads confidential file: allowed */
    check_mac(1, 2, "read");    /* public user reads confidential file: denied */
    check_mac(3, 1, "write");   /* secret user writes to public file: denied */

    return 0;
}
```

9 Access Control - Steps to Run

1. Save the code. Open a terminal and create the file:

```
nano access_control.c
```

Paste the program, then press Ctrl+O, Enter to save, and Ctrl+X to exit.

2. Compile.

```
gcc access_control.c -o access_control
```

If gcc is not installed, run `sudo apt install gcc` first (Ubuntu/Debian).

3. Run.

```
./access_control
```

The `./` tells the shell to run the program from the current folder.

4. Check the output. You should see the DAC permissions of testfile.txt, then the RBAC and MAC results. The DAC part shows:

```
Owner : rwx
Group : r-x
Others: r--
```

10 File I/O - Text and Binary Files

```
#include <stdio.h>

#define TEXT_FILE "student.txt"
#define BIN_FILE  "numbers.bin"

int main() {
    FILE *fp;
    char line[100], name[30], ch;
    int roll, nums[5] = {10, 20, 30, 40, 50}, back[5], i;

    /* ---------- WRITE TO A TEXT FILE ---------- */
    fp = fopen(TEXT_FILE, "w");
    if (fp == NULL) {
        printf("Error: cannot create %s\n", TEXT_FILE);
        return 1;
    }
    fputs("Student Record File\n", fp);     /* fputs() */
    fprintf(fp, "%d %s\n", 101, "Arun");    /* fprintf() */
    fputs("Grade: ", fp);
    fputc('A', fp);                         /* fputc() */
    fputc('\n', fp);
    fclose(fp);
    printf("Text file written using fputs(), fprintf(), fputc()\n\n");

    /* ---------- READ FROM A TEXT FILE ---------- */
    fp = fopen(TEXT_FILE, "r");
    if (fp == NULL) {
        printf("Error: cannot open %s\n", TEXT_FILE);
        return 1;
    }
    fgets(line, sizeof(line), fp);          /* fgets() reads a whole line */
    printf("Header line (fgets): %s", line);
    fscanf(fp, "%d %s", &roll, name);       /* fscanf() reads formatted data */
    printf("Roll number (fscanf): %d\n", roll);
    printf("Name        (fscanf): %s\n", name);
    fgetc(fp);                              /* skip the newline after the name */
    printf("Remaining text (fgetc):\n");
    while ((ch = fgetc(fp)) != EOF)         /* fgetc() reads one character */
        putchar(ch);
    fclose(fp);

    /* ---------- WRITE TO A BINARY FILE ---------- */
    fp = fopen(BIN_FILE, "wb");
    if (fp == NULL) {
        printf("Error: cannot create %s\n", BIN_FILE);
        return 1;
    }
    fwrite(nums, sizeof(int), 5, fp);       /* fwrite() */
    fclose(fp);
    printf("\nBinary file written using fwrite()\n");

    /* ---------- READ FROM A BINARY FILE ---------- */
    fp = fopen(BIN_FILE, "rb");
    if (fp == NULL) {
        printf("Error: cannot open %s\n", BIN_FILE);
        return 1;
    }
    fread(back, sizeof(int), 5, fp);        /* fread() */
    fclose(fp);
    printf("Numbers read back using fread(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", back[i]);
    printf("\n");

    return 0;
}
```